

# A NOITE LONGA

Como o código funciona — explicação completa

Jogo Narrativo Interativo em Java  
EXA863 — MI Programação — Fase 1  
Universidade Estadual de Feira de Santana

Documento de estudo — explica cada classe, cada conceito de Java usado,  
e o caminho completo de uma jogada dentro do código.

## COMO LER ESTE DOCUMENTO

Ele assume que você **não sabe nada de Java**. Cada conceito é explicado na primeira vez que aparece, com uma analogia e um exemplo do próprio jogo.

Se você já entende alguma parte, pule. A **Parte 5** (uma jogada passo a passo) é a que amarra tudo — se for ler só uma, leia aquela.

## Sumário

1. **O que o programa faz** — a visão de um minuto
2. **Os conceitos de Java** — classe, objeto, método, herança, enum, listas
3. **MVC** — por que o código está dividido em três pastas
4. **Classe por classe** — o que cada uma guarda e por quê
5. **Uma jogada passo a passo** — do teclado até a tela
6. **Como a história foi montada** — as duas passadas
7. **Requisito por requisito** — o que o enunciado pede e onde está
8. **Glossário** — todo termo técnico, em uma linha
9. **Perguntas da banca** — com as respostas

# 1. O que o programa faz

É um jogo de texto. O programa escreve na tela, o jogador digita um número, e a história continua de um jeito ou de outro conforme o número digitado.

Do lado de fora, é isto:

```
=====
Capítulo 1 - A Porta
=====

A chave ainda está na fechadura. Otávio não tirou.

Otávio: Se você sair, eu tranco. Não é ameaça, é o que se faz.

[ Fôlego 50 | Nervo 50 | Lucidez 50 ] Mochila: Terço da mãe;

[1] Sair. Não discutir.
[2] Pedir que ele deixe a porta destrancada.
[-] Chamar ele pra ir junto. (você não tem coragem de pedir)

O que você faz? _
```

Repare em três coisas nessa tela, porque elas resumem o jogo inteiro:

- As opções **[1]** e **[2]** estão disponíveis.
- A opção **[-]** está *bloqueada*: ela existe, mas o jogador não atende ao requisito (o Nervo dele é 50, e ela pede 55).
- Aquele **Fôlego / Nervo / Lucidez** e a **Mochila** mudam conforme o que o jogador escolhe.

## A ideia central, em três frases

**1. A história é um mapa pronto.** Quando o jogador aperta "nova partida", o programa já criou na memória todas as 28 cenas do jogo, com todas as falas e todas as escolhas. Ele não inventa nada enquanto roda — ele apenas *caminha* por esse mapa.

**2. Existem duas coisas separadas: o conteúdo e o estado.**

|                 | O que é                                             | Muda durante o jogo?           |
|-----------------|-----------------------------------------------------|--------------------------------|
| <b>Conteúdo</b> | Os capítulos, as falas, as escolhas                 | Nunca. É igual em toda partida |
| <b>Estado</b>   | Onde você está, seus atributos, quem confia em você | O tempo todo                   |

## ANALOGIA

O conteúdo é o **livro**. O estado é o **marcador de página** mais tudo o que você carrega na mochila. Duas pessoas podem ler o mesmo livro e estar em páginas diferentes, com mochilas diferentes.

**3. Cada escolha carrega dois pacotes:** as *condições* (o que precisa ser verdade pra ela aparecer) e as *consequências* (o que ela muda quando é escolhida).

## Os números do projeto

| Item                                               | Quantidade                                  |
|----------------------------------------------------|---------------------------------------------|
| Capítulos                                          | 10                                          |
| Cenas (capítulos + continuações + finais + mortes) | 28                                          |
| Escolhas                                           | 55                                          |
| Personagens com participação significativa         | 5 (Otávio, Davi, Antônio, Zulmira, Manuela) |
| Atributos do protagonista                          | 3 (Fôlego, Nervo, Lucidez)                  |
| Itens                                              | 9                                           |
| Flags (decisões memorizadas)                       | 13                                          |
| Finais                                             | 3 + 8 mortes (game over)                    |
| Classes Java                                       | 17                                          |

## 2. Os conceitos de Java

Esta parte explica cada ideia da linguagem que aparece no projeto. Se você já conhece alguma, pule — mas leia a de *enum*, *Map* e *Set*, porque são as que mais confundem quem está começando.

### 2.1 Classe e objeto

Uma **classe** é uma descrição. Um **objeto** é uma coisa concreta feita a partir dessa descrição.

#### ANALOGIA

A classe é a **planta de uma casa**. O objeto é **uma casa construída**. Com uma planta você constrói quantas casas quiser, e cada casa pode ter a parede de uma cor diferente.

No projeto, `Npc` é uma classe: ela descreve "um personagem que tem nome e confiança". Otávio, Davi e Zulmira são três **objetos** dessa classe — cada um com seu nome e sua própria confiança.

```
Npc otavio = new Npc("Otávio", 50);  
Npc davi   = new Npc("Davi", 30);  
Npc zulmira = new Npc("Dona Zulmira", 40);
```

A palavra `new` é o que constrói o objeto. Sem `new`, você tem só a planta.

#### ERRO COMUM

Escrever `Npc.getConfianca()` (com a classe) em vez de `otavio.getConfianca()` (com o objeto). É como perguntar a cor da parede *para a planta* em vez de para a casa. Java reclama com `non-static method cannot be referenced from a static context`.

### 2.2 Atributo e método

Dentro de uma classe existem duas coisas:

- **Atributos** (ou campos): o que o objeto *guarda*. São variáveis.
- **Métodos**: o que o objeto *sabe fazer*. São blocos de código com nome.

```
public class Npc extends PersonagemBase {

    private int confianca = 50;           // ATRIBUTO: guarda um número

    public int getConfianca() {           // MÉTODO: devolve esse número
        return this.confianca;
    }

    public void alterarConfianca(int variacao) { // MÉTODO: muda o número
        this.confianca = limitar(this.confianca + variacao);
    }
}
```

Lendo a linha do método com calma:

| Pedaco                        | Significa                                                                           |
|-------------------------------|-------------------------------------------------------------------------------------|
| <code>public</code>           | Qualquer classe pode chamar este método                                             |
| <code>void</code>             | Ele não devolve nada. ( <code>int</code> devolveria um número)                      |
| <code>alterarConfianca</code> | O nome                                                                              |
| <code>(int variacao)</code>   | Ele recebe um número, e dentro do método esse número se chama <code>variacao</code> |

A palavra `this` significa "deste objeto aqui". `this.confianca` é a confiança *deste* NPC, não a de outro.

## 2.3 Construtor

O **construtor** é um método especial que roda uma vez, no momento em que o objeto nasce. Ele tem o mesmo nome da classe e não declara tipo de retorno.

```
public Npc(String nome, int confianca) {
    super(nome);           // manda o nome para a classe pai
    this.confianca = limitar(confianca);
}
```

Serve para garantir que o objeto já nasça completo. Sem construtor, você poderia criar um NPC sem nome — e o programa quebraria depois, longe da causa.

## 2.4 `public` e `private` — encapsulamento

Esses dois marcadores dizem *quem pode enxergar* cada coisa.

| Marcador               | Quem pode acessar                 |
|------------------------|-----------------------------------|
| <code>public</code>    | Qualquer classe do projeto        |
| <code>private</code>   | Só a própria classe               |
| <code>protected</code> | A própria classe e as filhas dela |

A regra prática: **atributo é private, método é public**. Isso se chama **encapsulamento**, e existe por um motivo bem concreto.

Olha o que aconteceria se `confianca` fosse `public`:

```
otavio.confianca = 999;      // ninguém impede
otavio.confianca = -40;     // ninguém impede
```

Com ela `private`, o único caminho é o método:

```
otavio.alterarConfianca(999); // vira 100, porque o método aplica o limite
```

### A FRASE PARA A DEFESA ORAL

"A confiança é privada e só muda por `alterarConfianca`, que aplica o limite de 0 a 100. Como existe um caminho único, não há como um valor inválido entrar no sistema."

## Getter e setter

Um **getter** é um método que só devolve o valor de um atributo (`getConfianca`). Um **setter** é um que só troca o valor (`setConfianca`).

No nosso `Npc` tem getter mas **não** tem setter — de propósito. Um setter aceitaria qualquer número. O `alterarConfianca` recebe a *variação* ("+10", "-20") e aplica o limite. É a diferença entre "passe a valer 90" e "aumente 10".

## 2.5 Herança

**Herança** é quando uma classe aproveita o que outra já tem. A palavra é `extends`.

```
public abstract class PersonagemBase {
    private String nome;
    public String getNome() { return nome; }
}

public class Npc extends PersonagemBase { ... }
public class Protagonista extends PersonagemBase { ... }
```

`Npc` e `Protagonista` não declaram `nome` nem `getNome()` — eles herdam os dois. Escrever a mesma coisa em dois lugares seria duplicação, e duplicação é onde os bugs moram.

A relação se lê com o verbo **ser**: um NPC *é um* personagem. Se você não consegue dizer "X é um Y" com naturalidade, provavelmente não é herança.

### A palavra `abstract`

`PersonagemBase` é declarada `abstract`. Isso significa: **não dá para criar um objeto dela**.

```
PersonagemBase p = new PersonagemBase("Fulano"); // ERRO de compilação
```

Por quê? Porque "um personagem genérico" não existe no jogo. Ou é o protagonista, ou é um NPC. Marcar abstrata transforma esse erro conceitual em erro de compilação — o computador te avisa antes de você rodar.

### ANALOGIA

"Veículo" é abstrato: você não compra um veículo genérico, compra um carro ou uma moto. Mas todo veículo tem placa — e é isso que a classe abstrata guarda.

## Polimorfismo

Palavra grande, ideia simples: **tratar coisas diferentes do mesmo jeito**.

A classe `Dialogo` guarda quem falou:

```
private PersonagemBase personagemBase;
```

Repare que o tipo é `PersonagemBase`, não `Npc` nem `Protagonista`. Isso permite guardar qualquer um dos dois no mesmo campo. E na hora de imprimir:

```
System.out.println(fala.getPersonagem().getNome() + ": " + fala.getTexto());
```

A view não pergunta "isso é NPC ou protagonista?". Ela só pede o nome, e cada objeto responde o seu. **Isso é polimorfismo**, e é o motivo de a herança existir aqui: sem ela, o campo do `Dialogo` não teria tipo possível.

## 2.6 Enum

Um **enum** é uma lista fechada de valores possíveis, com nome.

```
public enum Atributo {
    FOLEGO("Fôlego"),
    NERVO("Nervo"),
    LUCIDEZ("Lucidez");

    private String nomeExibicao;

    Atributo(String nomeExibicao) {
        this.nomeExibicao = nomeExibicao;
    }

    public String getNomeExibicao() {
        return nomeExibicao;
    }
}
```

Existem exatamente três atributos no jogo, e ponto. Um enum diz isso ao compilador.

## POR QUE NÃO USAR STRING?

Podia-se escrever "folego" em todo lugar. O problema aparece assim:

```
alterarAtributo("fôlego", 10); // com acento
alterarAtributo("folego", 10); // sem acento
alterarAtributo("Folego", 10); // com maiúscula
```

Três strings diferentes, e o programa **compila** nas três. O bug só aparece rodando, e é silencioso: o atributo simplesmente não muda. Com enum, `Atributo.FOLEGO` não compila — o erro aparece na hora de escrever.

O detalhe do ("Fôlego") ao lado de cada valor: o enum pode carregar informação. Aqui ele carrega o nome bonito para mostrar na tela, então em vez de aparecer `FOLEGO` para o jogador, aparece `Fôlego`.

O projeto tem quatro enums:

| Enum                     | Para que serve                                       |
|--------------------------|------------------------------------------------------|
| <code>Atributo</code>    | Os três atributos do protagonista                    |
| <code>Item</code>        | Os nove itens que podem ir para a mochila            |
| <code>Flag</code>        | As decisões que o jogo precisa lembrar               |
| <code>Personagens</code> | A identidade de cada personagem e o nome de exibição |

## 2.7 List, Map e Set

São três formas de guardar *várias* coisas. A diferença é o que cada uma garante.

### List — uma fila ordenada

```
private List<Escolha> opcoes = new ArrayList<>();

opcoes.add(escolha1); // adiciona no fim
opcoes.get(0);        // pega a primeira (contagem começa em ZERO)
opcoes.size();        // quantas tem
```

O `<Escolha>` diz o que cabe dentro: só objetos `Escolha`. Se alguém tentar guardar outra coisa, não compila.

#### A CONTAGEM COMEÇA EM ZERO

A primeira posição é `get(0)`, não `get(1)`. É por isso que o controller faz `disponiveis.get(numero - 1)`: o jogador digita 1 pensando na primeira opção, e o programa precisa da posição 0.

## Map — um dicionário

Guarda pares "chave → valor". No projeto, guarda as cenas pelo id:

```
private Map<String, Cena> cenas = new HashMap<>();

cenas.put("CAP01", umaCena);      // guarda
cenas.get("CAP01");               // busca – instantâneo
```

### ANALOGIA

É a agenda do celular. Você não percorre os contatos um por um procurando o número: digita o nome e ele aparece. A chave é o nome, o valor é o número.

## Set — um conjunto sem repetição

```
private Set<Flag> flags = new HashSet<>();

flags.add(Flag.PORTA_ABERTA);      // liga
flags.contains(Flag.PORTA_ABERTA); // pergunta: true ou false
```

Um `Set` só responde "está dentro ou não está". Adicionar duas vezes a mesma coisa não faz diferença. É exatamente o que precisamos das flags: a porta ou foi deixada destrancada, ou não foi.

## 2.8 O laço `while`

Repete um bloco enquanto uma condição for verdadeira.

```
while (partida.getCenaAtual() != null) {
    ... mostra a cena, lê a escolha, anda ...
}
```

Lê-se: "enquanto a cena atual não for nula, faça isso". Quando o jogo chega num final, a cena não tem escolhas, o programa dá `break` (que significa "saia do laço agora") e o jogo termina.

## 2.9 O `switch`

Escolhe um caminho entre vários, conforme um valor.

```
switch (escolha) {
    case 1: jogar(...); break;
    case 2: menuInicial.exibirInstrucoes(); break;
    case 3: menuInicial.exibirCreditos(); break;
    case 4: rodando = false; break;
    default: menuInicial.exibirMensagem("Opção inválida!"); break;
}
```

O `break` no fim de cada caso é obrigatório: sem ele, o Java continua executando o caso seguinte. É um erro clássico e silencioso.

O `default` é o "se nenhum dos anteriores".

## 2.10 try e catch

Serve para lidar com erros que acontecem em tempo de execução, sem o programa morrer.

```
try {
    int opcao = teclado.nextInt();    // pode dar erro se o usuário digitar letra
    ...
} catch (InputMismatchException e) {
    teclado.nextLine();              // limpa o que não era número
    System.out.println("Isso não é um número.");
}
```

Traduzindo: "tente fazer isso; se der esse erro específico, faça aquilo em vez de quebrar".

## 2.11 static final

```
public static final int MINIMO = 0;
public static final int MAXIMO = 100;
```

| Palavra             | Significa                                                         |
|---------------------|-------------------------------------------------------------------|
| <code>static</code> | Pertence à classe, não a cada objeto. Existe um só, compartilhado |
| <code>final</code>  | Não pode mudar depois de definido. É uma constante                |

Serve para dar nome a um número mágico. `MAXIMO` é mais legível que `100` espalhado por cinco lugares — e se um dia a escala mudar, muda em um lugar só.

## 2.12 Scanner

É a classe do Java que lê o que o usuário digita.

```
Scanner teclado = new Scanner(System.in);

int numero = teclado.nextInt();    // lê um número
String texto = teclado.nextLine(); // lê uma linha inteira
```

### A ARMADILHA DO NEXTINT SEGUIDO DE NEXTLINE

Quando você digita `5` e aperta ENTER, chegam **dois** caracteres: o `5` e a quebra de linha. O `nextInt()` consome só o `5` e **deixa a quebra de linha esperando**.

Aí o próximo `nextLine()` encontra essa quebra pendente e devolve texto vazio na hora, sem deixar você digitar. Por isso, no projeto, todo `nextInt()` é seguido de um `nextLine()` que serve só para limpar.

### **POR QUE EXISTE UM SCANNER SÓ NO PROJETO**

Cada Scanner mantém um buffer interno — uma "caixinha" onde ele adianta a leitura. Com dois Scanners lendo o mesmo teclado, o primeiro pode puxar texto que era do segundo, e o segundo estoura com `NoSuchElementException`.

O pior é que isso **depende de velocidade**: digitando devagar funciona, e alimentando rápido (que é como um teste automatizado faz) quebra. Por isso o `JogoController` cria um Scanner e empresta o mesmo para as duas views.

## 3. MVC — por que três pastas

MVC significa **Model, View, Controller**. É uma forma de organizar o programa separando três responsabilidades que costumam se misturar.

| Camada            | Responsabilidade                                             | No nosso projeto                          |
|-------------------|--------------------------------------------------------------|-------------------------------------------|
| <b>Model</b>      | Os dados e as regras do jogo                                 | Cena, Escolha, Npc, Partida, Historia ... |
| <b>View</b>       | Mostrar na tela e ler o teclado                              | MenuInicial, ExibirJogo                   |
| <b>Controller</b> | Coordenar: perguntar, mandar aplicar, decidir a próxima tela | JogoController                            |

### ANALOGIA: UM RESTAURANTE

- **Model** — a cozinha e os ingredientes. Sabe fazer comida, nunca fala com o cliente.
- **View** — o cardápio e o prato na mesa. Só mostra, não cozinha.
- **Controller** — o garçom. Leva o pedido pra cozinha e o prato pra mesa. Não cozinha nem come.

## A regra de ouro: a direção das setas

```
Main → JogoController → MenuInicial / ExibirJogo (views)
                               → Historia / Partida / Cena (model)

View → Lê o model (nunca altera)
Model → não conhece ninguém
```

Duas coisas nunca podem acontecer:

- **O model imprimir na tela.** Se aparecer `System.out.println` dentro de `Npc` ou `Cena`, quebrou. Numa Fase 3 com interface gráfica, esse `println` viraria lixo.
- **A view conhecer o controller.** Se `ExibirJogo` tiver `import controller...`, quebrou. A view é burra de propósito: alguém manda ela exibir, e ela exibe.

Nota: a view *lê* classes do model (ela recebe uma `Cena` para poder mostrá-la). Isso é normal e esperado. O proibido é ela *alterar* — nada de `partida.ligarFlag(...)` dentro de uma classe da view.

## Por que isso importa na prática

Três motivos concretos, nenhum deles teórico:

1. **Fase 3.** Quando virar JavaFX, você escreve views novas e o model não muda uma linha.

2. **Testes de unidade.** Dá para testar `escolha.aplicar(partida)` sem console e sem teclado. Se a regra estivesse dentro da view, seria impossível.
3. **Trabalho em dupla.** Uma pessoa mexe na view enquanto a outra mexe no model, sem conflito no Git.

## 4. Classe por classe

Aqui cada classe é explicada: o que guarda, o que faz, e *por que* foi feita assim. As decisões de projeto estão marcadas, porque são elas que a banca pergunta.

### 4.1 `PersonagemBase` — a raiz da família

```
public abstract class PersonagemBase {  
    private String nome;  
  
    public PersonagemBase(String nome) {  
        this.nome = nome;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
}
```

Cinco linhas úteis, três decisões dentro delas:

- **`abstract`** — não existe "personagem genérico" no jogo. Marcar abstrata impede `new PersonagemBase(...)` no compilador.
- **`private String nome`** — ninguém de fora troca o nome.
- **Só tem o que *todos* compartilham** — por isso a confiança **não** está aqui. Se estivesse, o protagonista teria uma "confiança em relação a si mesmo", que não significa nada.

#### REGRA QUE A BANCA COBRA

Um campo que não faz sentido para metade das subclasses subiu alto demais na hierarquia. Nome faz sentido para os dois; confiança só faz para o NPC.

## 4.2 Protagonista — quem o jogador controla

```
public class Protagonista extends PersonagemBase {

    public static final int MINIMO = 0;
    public static final int MAXIMO = 100;

    private int folego = 50;
    private int nervo = 50;
    private int lucidez = 50;

    public void alterarAtributo(Atributo atributo, int variacao) {
        switch (atributo) {
            case FOLEGO: folego = limitar(folego + variacao); break;
            case NERVO: nervo = limitar(nervo + variacao); break;
            case LUCIDEZ: lucidez = limitar(lucidez + variacao); break;
        }
    }

    public int getAtributo(Atributo atributo) {
        switch (atributo) {
            case FOLEGO: return folego;
            case NERVO: return nervo;
            case LUCIDEZ: return lucidez;
            default: return 0;
        }
    }

    private int limitar(int valor) {
        if (valor < MINIMO) return MINIMO;
        if (valor > MAXIMO) return MAXIMO;
        return valor;
    }
}
```

### **Por que um método só em vez de três setters**

A versão intuitiva seria `setFolego`, `setNervo`, `setLucidez`. Dois problemas com isso:

**Primeiro:** as consequências do jogo pensam em variação (" $+10$ "), não em valor final. Com setter, quem chama teria que escrever `p.setFolego(p.getFolego() + 10)` — aritmética espalhada por todo o projeto.

**Segundo, e mais importante:** onde ficaria a regra do limite 0-100? Com três setters, ou você repete o limite três vezes, ou confia que quem chama vai lembrar. Com um método só, o limite mora dentro dele e **não existe outro caminho**.

O `limitar` é `private` porque é assunto interno da classe. Ninguém de fora precisa saber que ele existe.

## 4.3 `Npc` — os outros personagens

```
public class Npc extends PersonagemBase {  
  
    private int confianca = 50;  
  
    public Npc(String nome) {  
        super(nome);  
    }  
  
    public Npc(String nome, int confianca) {  
        super(nome);  
        this.confianca = limitar(confianca);  
    }  
  
    public int getConfianca() {  
        return this.confianca;  
    }  
  
    public void alterarConfianca(int variacao) {  
        this.confianca = limitar(this.confianca + variacao);  
    }  
}
```

Duas coisas para notar:

**Dois construtores.** Java permite vários métodos com o mesmo nome desde que os parâmetros sejam diferentes — chama-se *sobrecarga*. Um cria o NPC com confiança padrão (50), o outro deixa escolher. Por isso Davi começa em 30 e Otávio em 50: o irmão já confia mais em você do que um menino que acabou de te conhecer.

`super(nome)` manda o nome para o construtor da classe pai. Tem que ser a primeira linha do construtor — o pai precisa estar montado antes de o filho começar.

## 4.4 `Dialogo` — uma fala

```
public class Dialogo {  
    private PersonagemBase personagemBase;  
    private String texto;  
}
```

Guarda duas coisas: *quem* falou e *o quê*. É a classe mais simples do projeto, e é ela que justifica a herança existir — o campo é do tipo `PersonagemBase` justamente para caber tanto um NPC quanto o protagonista.

## 4.5 **Cena** — um capítulo

```
public class Cena {
    private String id;           // "CAP01" -- identificação interna
    private String titulo;       // "Capítulo 1 - A Porta"
    private String textoPrincipal; // a narração
    private List<Escolha> opcoes; // as alternativas
    private List<Dialogo> dialogos; // as falas
}
```

Repare que o construtor **cria as duas listas vazias**:

```
public Cena(String id, String titulo, String textoPrincipal) {
    this.id = id;
    this.titulo = titulo;
    this.textoPrincipal = textoPrincipal;
    this.opcoes = new ArrayList<>(); // lista vazia, não nula
    this.dialogos = new ArrayList<>();
}
```

### **POR QUE ISSO EVITA O ERRO MAIS COMUM DE JAVA**

Sem essas duas linhas, as listas seriam `null` — que significa "não aponta para nada". Aí a primeira vez que alguém fizesse `cena.getOpcoes().add(...)`, o programa quebraria com `NullPointerException`. Inicializar no construtor garante que a lista *existe*, mesmo vazia.

### ***Uma cena sem escolhas é um final***

Esse é um detalhe elegante do desenho: os finais e as mortes são cenas normais que simplesmente não recebem nenhuma escolha. O laço do jogo vê a lista vazia e encerra. Não precisou de nenhum campo `isFinal` nem de classe especial.

## 4.6 **Escolha** — o coração do jogo

É a maior classe do model, e é onde estão os requisitos mais pesados do enunciado. Ela tem três blocos de campos.

### **Bloco 1 — identificação**

```
private String id;           // "ESC0102"
private String textoExibido; // "Pedir que ele deixe a porta destrancada."
private Cena cenaDestino;    // pra onde vai depois
```

## Bloco 2 — consequências (o que muda)

```
private NPC npc;                // quem tem a confiança alterada
private int consequencia;       // quanto muda
private Atributo atributoAfetado;
private int deltaAtributo;
private Item itemGanho;
private Item itemPerdido;
private Flag flagLigada;
```

## Bloco 3 — condições (o que precisa ser verdade)

```
private Atributo atributoExigido;
private int valorMinimoAtributo;
private Item itemExigido;
private NPC npcConfianca;
private int confiancaMinima;
private Flag flagExigida;
private Flag flagProibida;
private String motivoDoBloqueio;
```

### "POR QUE TANTOS CAMPOS?" — A PERGUNTA DA BANCA

Porque são **opcionais**. Uma escolha usa dois ou três; o resto fica nulo (ou zero) e simplesmente não é verificado nem aplicado. É a solução mais direta para o requisito de consequências e condições, sem criar dezenas de classes.

## Os métodos de configuração

```
public Escolha comConfianca(NPC npc, int variacao) {
    this.npc = npc;
    this.consequencia = variacao;
    return this;                // <-- repare aqui
}
```

Aquele `return this` é o truque que deixa a `Historia` legível. Como o método devolve o próprio objeto, dá para chamar o próximo em seguida:

```
new Escolha("ESC0102", "Pedir que ele deixe a porta destrancada.", getCena("CAP02"))
    .ligaFlag(Flag.PORTA_ABERTA)
    .comConfianca(otavio, 10)
    .comAtributo(Atributo.LUCIDEZ, 5);
```

Sem isso, seria preciso guardar em variável e chamar linha por linha, 55 vezes. Esse padrão tem nome: **interface fluente** (ou *method chaining*). É o mesmo que o `StringBuilder` do Java faz.

## O método `estaDisponivel` — condições de acesso

```
public boolean estaDisponivel(Partida partida) {  
    if (atributoExigido != null  
        && partida.getProtagonista().getAtributo(atributoExigido) <  
        valorMinimoAtributo) {  
        return false;  
    }  
    if (itemExigido != null && !partida.getInventario().possui(itemExigido)) {  
        return false;  
    }  
    if (npcConfianca != null && npcConfianca.getConfianca() < confiancaMinima) {  
        return false;  
    }  
    if (flagExigida != null && !partida.temFlag(flagExigida)) {  
        return false;  
    }  
    if (flagProibida != null && partida.temFlag(flagProibida)) {  
        return false;  
    }  
    return true;  
}
```

A lógica é: **qualquer condição que falhe já devolve `false`**. Se o método chegou até o fim sem sair, nenhuma falhou, e a escolha aparece.

Traduzindo o primeiro `if`: "se existe um atributo exigido e o valor do protagonista é menor que o mínimo, então bloqueia". A verificação `!= null` vem antes porque a maioria das escolhas não exige atributo nenhum — esse campo fica nulo e o `if` é ignorado.

Os símbolos: `&&` é "e", `!` é "não". Então `!possui(item)` lê-se "não possui o item".

## O método `aplicar` — consequências

```
public String aplicar(Partida partida) {
    String aviso = "";

    if (npc != null && consequencia != 0) {
        npc.alterarConfianca(consequencia);
        aviso += npc.getNome() + (consequencia > 0 ? " confia mais em você. "
            : " confia menos em você. ");
    }
    if (atributoAfetado != null && deltaAtributo != 0) {
        partida.getProtagonista().alterarAtributo(atributoAfetado, deltaAtributo);
        aviso += atributoAfetado.getNomeExibicao() + " "
            + (deltaAtributo > 0 ? "+" : "") + deltaAtributo + ". ";
    }
    if (itemGanho != null) {
        partida.getInventario().adicionar(itemGanho);
        aviso += "Você guardou: " + itemGanho.getNomeExibicao() + ". ";
    }
    if (itemPerdido != null) {
        partida.getInventario().remover(itemPerdido);
    }
    if (flagLigada != null) {
        partida.ligarFlag(flagLigada);
        // silenciosa de propósito: o efeito aparece capítulos depois
    }

    return aviso.isEmpty() ? null : aviso.trim();
}
```

Cada `if` cuida de um tipo de efeito. O que estiver nulo é pulado.

O `+=` vai juntando texto: `aviso` começa vazio e cada efeito acrescenta uma frase. No fim, se ficou vazio, devolve `null` — e a view sabe que não tem nada para mostrar.

Aquele `condicao ? valorA : valorB` é o **operador ternário**: uma forma curta de `if/else` dentro de uma expressão. Lê-se "se a condição for verdadeira, use A; senão, use B".

### DECISÃO DE PROJETO IMPORTANTE

É a **própria escolha** que aplica seus efeitos, não o controller. Isso significa que dá para testar uma consequência assim: cria uma `Partida`, chama `escolha.aplicar(partida)`, confere o resultado. Três linhas, sem console, sem menu, sem simular teclado. É o que torna os testes de unidade do enunciado viáveis.

## 4.7 Partida — o estado do jogo

```
public class Partida {

    private Protagonista protagonista;
    private Inventario inventario;
    private Cena cenaAtual;
    private Set<Flag> flags;

    private Npc otavio;
    private Npc mae;
    private Npc davi;
    private Npc zulmira;
    private Npc antonio;
    private Npc homemDeTerno;
    private Npc manuela;

    public Partida(String nomeJogador) {
        this.protagonista = new Protagonista(nomeJogador);
        this.inventario = new Inventario();
        this.flags = new HashSet<>();

        this.otavio = new Npc("Otávio", 50);
        this.davi = new Npc("Davi", 30);
        ...
        this.inventario.adicionar(Item.TERC0);
    }

    public void ligarFlag(Flag flag) { flags.add(flag); }
    public boolean temFlag(Flag flag) { return flags.contains(flag); }
}
```

Esta é a classe mais importante do projeto, e ela quase não tem lógica — ela **guarda**.

### O que ela resolve

**1. Junta o estado que estaria espalhado.** Sem ela, o protagonista, os NPCs, as flags e o inventário seriam variáveis soltas dentro de um método do controller.

**2. Faz a partida nova começar limpa.** Este é o ponto mais sutil e o mais importante. Os NPCs nascem **dentro do construtor da Partida**. Então:

```
Partida partida = new Partida(nomeProtagonista); // NPCs novos, confiança zerada
```

Terminar o jogo e começar de novo cria uma `Partida` nova, com NPCs novos. Nada da partida anterior sobrevive. Se os NPCs fossem campos da `Historia`, eles seriam criados uma vez só e a confiança da partida passada continuaria lá.

**3. Prepara a Fase 2.** Salvar o jogo vai ser gravar a `Partida`: o id da cena atual, os atributos, as confianças, as flags, os itens. A `Historia` não precisa ser salva, porque não muda.

### A `cenaAtual` é só um marcador

Vale insistir nisso porque confunde: a `Partida` **não guarda os capítulos**. Ela guarda uma referência a *um* deles — em qual o jogador está agora. Os 28 capítulos são da `Historia`.

## 4.8 Inventario — a mochila

```
public class Inventario {  
  
    private boolean temTercoDaMae = false;  
    private boolean temRadioDePilha = false;  
    private boolean temMedalhaSaoJorge = false;  
    ...  
  
    public boolean possui(Item item) {  
        switch (item) {  
            case TERC0:    return temTercoDaMae;  
            case RADIO:    return temRadioDePilha;  
            case MEDALHA: return temMedalhaSaoJorge;  
            ...  
        }  
    }  
  
    public void adicionar(Item item) { definir(item, true); }  
    public void remover(Item item)  { definir(item, false); }  
}
```

A ideia é simples: cada item é um `boolean` — tem ou não tem. Os métodos `possui` e `adicionar` traduzem o enum `Item` para o campo certo.

**O ganho disso:** o resto do jogo nunca precisa saber que por baixo são campos separados. A `Escolha` só chama `inventario.possui(Item.CHAVE)`. Se um dia vocês trocarem os booleanos por uma lista, nada fora da classe muda.

Isso tem nome: a classe **esconde a implementação atrás de uma interface pública**. É encapsulamento aplicado a uma estrutura de dados, e é um bom exemplo para o relatório.

## 4.9 Os quatro enums

| Enum        | Valores                | Onde é lido                   |
|-------------|------------------------|-------------------------------|
| Atributo    | FOLEGO, NERVO, LUCIDEZ | Protagonista, Escolha         |
| Item        | 9 itens                | Inventario, Escolha           |
| Flag        | 13 decisões            | Partida, Escolha              |
| Personagens | 9 nomes                | Partida (para nomear os NPCs) |

O `Flag` merece atenção, porque é ele que produz o requisito mais difícil do enunciado — as **consequências posteriores**.

```
public enum Flag {
    PORTA_ABERTA,      // ligada no capítulo 1, lida nos capítulos 5, 8 e 10
    SALVOU_DAVI,       // ligada no capítulo 2, lida no 5
    DAVI_JUNTO,        // ligada no capítulo 5, lida no 7
    OUVIU_RADIO,       // ligada no capítulo 4, lida no mesmo capítulo
    ...
}
```

Um enum sozinho só *lista* as flags possíveis. Quem guarda *quais aconteceram* é o `Set<Flag>` dentro da `Partida`.

## 4.10 História — o roteiro

É a maior classe do projeto (umas 900 linhas), e é grande porque **ela é o conteúdo**: os textos dos 10 capítulos estão nela. O que importa é que **nenhum método dela é grande** — a Parte 6 explica a organização.

O essencial: ela não guarda estado nenhum de partida. Ela só monta objetos e devolve a primeira cena.

## 4.11 As duas views

### MenuInicial

Cuida do menu, das instruções, dos créditos e de perguntar o nome. Tem as quatro opções que o enunciado exige.

### ExibirJogo

Cuida da tela do jogo: título, narração, falas, status e a lista de opções.

```
public void exibirCena(Cena cena, Partida partida,
    List<Escolha> disponiveis, List<Escolha> bloqueadas) {
```

Ela recebe as duas listas já separadas pelo controller. E imprime as bloqueadas também:

```
for (Escolha opcao : bloqueadas) {
    System.out.println("  [-] " + opcao.getTextoExibido()
        + " (" + opcao.getMotivoDoBloqueio() + ")");
}
```

Mostrar o bloqueio é uma decisão de design, não um acidente: o jogador precisa perceber que existia outro caminho e que uma decisão dele o fechou. É o que torna a consequência posterior **visível** — e visível é o que o professor consegue avaliar jogando.

Aquele `for (Escolha opcao : bloqueadas)` é o **for-each**: lê-se "para cada Escolha, que vou chamar de `opcao`, dentro de `bloqueadas`". É a forma limpa de percorrer uma lista inteira.

## 4.12 JogoController — o coordenador

```
private Scanner teclado = new Scanner(System.in);
private MenuInicial menuInicial = new MenuInicial(teclado);
private ExibirJogo exibirJogo = new ExibirJogo(teclado);
private Historia historia = new Historia();
```

O controller cria o Scanner e **empresta** para as duas views, passando no construtor delas. É assim que existe um Scanner só no programa inteiro.

Isso tem nome: **injeção de dependência**. Em vez de cada classe criar o que precisa, ela recebe pronto de fora. O ganho aqui é evitar o bug dos buffers; num projeto maior, o ganho é poder trocar a dependência por uma versão falsa num teste.

### O menu em laço

```
public void iniciarPartida() {
    boolean rodando = true;

    while (rodando) {
        int escolha = menuInicial.exibir();

        switch (escolha) {
            case 1: jogar(menuInicial.pedirNome()); break;
            case 2: menuInicial.exibirInstrucoes(); break;
            case 3: menuInicial.exibirCreditos(); break;
            case 4: rodando = false; break;
            default: menuInicial.exibirMensagem("Opção inválida!"); break;
        }
    }
}
```

Duas coisas importantes:

**Não existe `System.exit(0)`.** Sair é uma decisão — a variável `rodando` vira `false`, o laço termina, o método acaba, o programa acaba naturalmente. O `System.exit` mataria a JVM na hora, e num teste de unidade derrubaria o próprio executor de testes.

**O menu volta.** Quando `jogar(...)` termina, o laço recomeça e o menu aparece de novo — que é o que o enunciado pede ("retornar ao menu após o encerramento").

## O laço do jogo

```
private void jogar(String nomeProtagonista) {

    Partida partida = new Partida(nomeProtagonista);
    partida.setCenaAtual(historia.montarHistoria(partida));

    while (partida.getCenaAtual() != null) {

        Cena cena = partida.getCenaAtual();

        // separa as escolhas disponíveis das bloqueadas
        List<Escolha> disponiveis = new ArrayList<>();
        List<Escolha> bloqueadas = new ArrayList<>();
        for (Escolha opcao : cena.getOpcoes()) {
            if (opcao.estaDisponivel(partida)) {
                disponiveis.add(opcao);
            } else {
                bloqueadas.add(opcao);
            }
        }

        exibirJogo.exibirCena(cena, partida, disponiveis, bloqueadas);

        if (disponiveis.isEmpty()) {                // cena sem saída = fim
            exibirJogo.exibirMensagemFimDeJogo();
            break;
        }

        int numero = exibirJogo.pedirEscolhaJogador(disponiveis.size());
        Escolha escolhida = disponiveis.get(numero - 1);

        exibirJogo.exibirConsequencia(escolhida.aplicar(partida));

        partida.setCenaAtual(escolhida.getCenaDestino());
    }
}
```

São umas dez linhas de lógica, e elas são o jogo inteiro. Repare no que o controller **não** faz: ele não sabe o que é confiança, não sabe o que é flag, não decide o que uma escolha causa. Ele pergunta, manda aplicar e anda.

O `numero - 1` é a conversão entre o que o jogador vê (opção 1) e a posição na lista (índice 0).

## 5. Uma jogada passo a passo

Esta é a parte que amarra tudo. Vamos seguir **uma única escolha** desde o momento em que o jogador digita `2` até o efeito dela aparecer quatro capítulos depois.

Situação: capítulo 1, o jogador vai escolher "Pedir que ele deixe a porta destrancada".

### Passo 1 — O controller pega a cena atual

```
Cena cena = partida.getCenaAtual();
```

A `Partida` devolve o objeto `Cena` guardado no campo `cenaAtual`. Neste momento, é o objeto que foi criado lá na `Historia` com o id `"CAP01"`.

### Passo 2 — O controller separa as escolhas

```
for (Escolha opcao : cena.getOpcoes()) {  
    if (opcao.estaDisponivel(partida)) {  
        disponiveis.add(opcao);  
    } else {  
        bloqueadas.add(opcao);  
    }  
}
```

A cena tem três escolhas. Para cada uma, o controller pergunta `estaDisponivel(partida)`.

Vamos acompanhar a terceira, `ESC0103` ("Chamar ele pra ir junto"). Ela foi montada assim na `Historia`:

```
new Escolha("ESC0103", "Chamar ele pra ir junto.", getCena("CAP02"))  
    .exigeAtributo(Atributo.NERV0, 55, "você não tem coragem de pedir")
```

Dentro do `estaDisponivel`:

```
if (atributoExigido != null  
    && partida.getProtagonista().getAtributo(atributoExigido) < valorMinimoAtributo) {  
    return false;  
}
```

Substituindo pelos valores reais:

- `atributoExigido` é `NERV0` — não é nulo, então a segunda parte é verificada
- `getAtributo(NERV0)` devolve `50`
- `valorMinimoAtributo` é `55`
- `50 < 55` é verdadeiro → `return false`

Essa escolha vai para a lista de **bloqueadas**. As outras duas não têm condição nenhuma, então caem em **disponiveis**.

### Passo 3 — A view desenha a tela

```
[1] Sair. Não discutir.  
[2] Pedir que ele deixe a porta destrancada.  
[-] Chamar ele pra ir junto. (você não tem coragem de pedir)
```

As duas disponíveis ganham número; a bloqueada aparece com `[-]` e o motivo que foi escrito na hora de montar a escolha.

### Passo 4 — O jogador digita

```
int numero = exibirJogo.pedirEscolhaJogador(disponiveis.size());
```

O método recebe `2` (o tamanho da lista de disponíveis) e só aceita 1 ou 2. Se o jogador digitar 5 ou uma letra, ele avisa e pergunta de novo, num laço.

O jogador digita `2`. O método devolve `2`.

### Passo 5 — O controller acha o objeto

```
Escolha escolhida = disponiveis.get(numero - 1); // get(1)
```

Posição 1 da lista de disponíveis = a segunda = `ESC0102`.

### Passo 6 — A escolha aplica seus efeitos

```
exibirJogo.exibirConsequencia(escolhida.aplicar(partida));
```

A `ESC0102` foi montada com três consequências:

```
.ligaFlag(Flag.PORTA_ABERTA)  
.comConfianca(otavio, 10)  
.comAtributo(Atributo.LUCIDEZ, 5)
```

Dentro do `aplicar`, na ordem:

| Bloco                                | O que faz                                             | O que acrescenta ao aviso     |
|--------------------------------------|-------------------------------------------------------|-------------------------------|
| <code>npc != null</code>             | <code>otavio.alterarConfianca(10) → 50 vira 60</code> | "Otávio confia mais em você." |
| <code>atributoAfetado != null</code> | <code>alterarAtributo(LUCIDEZ, 5) → 50 vira 55</code> | "Lucidez +5."                 |
| <code>flagLigada != null</code>      | <code>partida.ligarFlag(PORTA_ABERTA)</code>          | nada — é silenciosa           |

O método devolve o texto juntado.

## Passo 7 — A view mostra o efeito

```
> Otávio confia mais em você. Lucidez +5.
```

```
[ENTER]
```

Se a escolha não tivesse nenhum efeito visível, `aplicar` devolveria `null` e a view não imprimiria nada.

## Passo 8 — O jogo anda

```
partida.setCenaAtual(escolhida.getCenaDestino());
```

A `ESC0102` tem como destino a cena `CAP02`. O campo `cenaAtual` passa a apontar para ela, o laço recomeça e o capítulo 2 aparece.

## E quatro capítulos depois...

No capítulo 5, existe esta escolha:

```
new Escolha("ESC0505", "Pegar a estrada da esquerda. Voltar pra casa.",
    getCena("FIM_DESISTE"))
    .exigeFlag(Flag.PORTA_ABERTA, "a porta está trancada")
```

Quando o controller perguntar `estaDisponivel(partida)`, o `estaDisponivel` vai fazer:

```
if (flagExigida != null && !partida.temFlag(flagExigida)) {
    return false;
}
```

E `partida.temFlag(PORTA_ABERTA)` devolve **true**, porque a flag foi ligada no passo 6, quatro capítulos atrás. A escolha aparece.

Para quem escolheu a opção 1 no capítulo 1, o `Set` de flags está vazio, e a tela mostra:

```
[ - ] Pegar a estrada da esquerda. Voltar pra casa. (a porta está trancada)
```

### ISTO É O REQUISITO MAIS DIFÍCIL DO ENUNCIADO, FUNCIONANDO

"Consequências: aplicar efeitos imediatos e **posteriores** às escolhas" e "Condições de acesso: bloquear ou liberar escolhas e cenas de acordo com itens, atributos, relacionamentos ou **decisões anteriores**." Uma decisão do capítulo 1 determina se um final existe no capítulo 5. E o jogador *vê* a porta que ele mesmo fechou.

## As outras cadeias de consequência

| Decisão                           | Onde  | Cobra em                                                                   |
|-----------------------------------|-------|----------------------------------------------------------------------------|
| Pedir a porta destrancada         | CAP 1 | CAP 5 e 8 (desistir), CAP 10 (uma fala com a Manuela)                      |
| Dar o terço ao Davi               | CAP 2 | CAP 7 — a confiança dele é o que permite pegar a medalha                   |
| Ajoelhar quando a Procissão passa | CAP 3 | CAP 3 — a Zulmira só ensina as regras a quem obedeceu                      |
| Ouvir a transmissão inteira       | CAP 4 | CAP 4 — sem ouvir, não dá pra avisar o Antônio, e sem avisar não tem rádio |
| Aceitar a vela apagada            | CAP 7 | CAP 8 — a vela no bolso abre a escolha que mata                            |
| Levar o Davi da ponte             | CAP 5 | CAP 7 — é o único jeito de pegar a medalha com fôlego baixo                |

A do Davi é a mais bonita mecanicamente. No capítulo 7 existem duas formas de pegar a medalha:

```
// forma 1: fôlego alto
.exigeAtributo(Atributo.FOLEGO, 55, "você não alcança sozinho")

// forma 2: o menino sobe nos seus ombros
.exigeFlag(Flag.DAVI_JUNTO, "Davi não está com você")
.exigeConfianca(davi, 60, "ele não confia em você o bastante")
```

Um jogador que gastou fôlego no caminho só consegue a medalha pela segunda — e ela depende de como ele tratou uma criança cinco capítulos antes.

## 6. Como a história foi montada

A classe `Historia` tem quase mil linhas. Isso assusta, mas ela é grande pelo motivo certo: **ela é o conteúdo**. Os textos dos dez capítulos estão dentro dela.

### O QUE "MÉTODOS COESOS" QUER DIZER NO ENUNCIADO

O enunciado cobra *métodos coesos*, não arquivos curtos. Uma classe de roteiro com 900 linhas é normal. Um *método* com 900 linhas seria o problema. Na nossa `Historia`, nenhum método passa de 45 linhas.

### 6.1 O problema que a montagem em duas passadas resolve

Para criar uma escolha, é preciso passar a cena de destino:

```
new Escolha("ESC0101", "Sair. Não discutir.", cap02)
```

Ou seja: o objeto `cap02` **já precisa existir** quando o capítulo 1 é montado. Se o código fosse escrito de forma ingênua, seria preciso criar o capítulo 2 antes do 1, o 3 antes do 2... e a história apareceria de trás para frente no arquivo. Com dez capítulos, você escreveria o final primeiro.

Pior: seria **impossível** uma cena apontar para trás, ou para si mesma.

### 6.2 A solução: criar todas vazias primeiro

```
public Cena montarHistoria(Partida partida) {
    cenas.clear();

    criarCenas();                // 1ª passada

    montarCap01(partida);        // 2ª passada
    montarCap02(partida);
    montarCap03(partida);
    ...
    montarCap10(partida);

    return getCena("CAP01");
}
```

## Primeira passada — `criarCenas()`

```
private void criarCenas() {
    criar("CAP01", "Capítulo 1 - A Porta", T_CAP01);
    criar("CAP02", "Capítulo 2 - A Rua de Casa", T_CAP02);
    criar("CAP02B", "Capítulo 2 - A Esquina", T_CAP02B);
    ...
    criar("FIM_A_TEMPO", "FIM", T_FIM_A_TEMPO);
    criar("MORTE_NOME", "GAME OVER", T_MORTE_NOME);
}
```

Vinte e oito linhas curtas. Cada cena nasce com id, título e narração — e **sem nenhuma escolha**. Nenhuma conhece nenhuma. Todas vão para o `Map` :

```
private void criar(String id, String titulo, String texto) {
    cenas.put(id, new Cena(id, titulo, texto));
}
```

## Segunda passada — um método por capítulo

Agora que todas existem, cada capítulo pode buscar qualquer outra pelo id:

```
private void montarCap01(Partida partida) {
    Cena cena = getCena("CAP01");
    NPC otavio = partida.getOtavio();

    falar(cena, otavio, "Não olha pra mim assim.");
    falar(cena, otavio, "Ela sabia. Ninguém sai na Noite Longa.");
    ...

    cena.getOpcoes().add(
        new Escolha("ESC0101", "Sair. Não discutir.", getCena("CAP02"))
            .comConfianca(otavio, -10)
            .comAtributo(Atributo.NERVO, 5));
    ...
}
```

Agora a história é escrita na ordem natural: capítulo 1, capítulo 2, capítulo 3.

## 6.3 Os textos como constantes

```
private static final String T_CAP01 =
    "A chave ainda está na fechadura. Otávio não tirou.\n\n" +
    "Ele está com a mão nela há dois minutos, do jeito de quem espera você\n" +
    "dizer alguma coisa que resolva. Você não tem essa coisa.";
```

Ficam no topo da classe. Sem isso, o `criarCenas()` teria vinte e oito blocos de texto no meio e seria ilegível. Assim, ele tem vinte e oito linhas curtas e o texto fica separado, como se fosse um arquivo de conteúdo dentro do código.

O `\n` é uma quebra de linha; `\n\n` pula uma linha em branco. O `+` junta as strings.

## 6.4 Por que a **Historia** recebe a **Partida**

```
public Cena montarHistoria(Partida partida)
```

Porque as escolhas precisam apontar para os objetos **Npc daquela partida** — são eles que têm a confiança que vai ser alterada.

```
Npc otavio = partida.getOtavio();           // o Otávio DESTA partida
...
.comConfianca(otavio, -10)                  // altera a confiança DESTA objeto
```

Cada partida nova cria NPCs novos, e a história é remontada apontando para eles. É o que garante que a confiança comece zerada em cada jogo.

## 6.5 As mortes e os finais

Não existe classe especial para eles. São cenas normais criadas **sem nenhuma escolha**:

```
criar("MORTE_NOME", "GAME OVER", T_MORTE_NOME);
```

E uma escolha que mata simplesmente aponta para lá:

```
new Escolha("ESC0201", "Olhar.", getCena("MORTE_NOME"))
```

Quando o jogador escolhe, o controller anda para essa cena, mostra o texto, vê que a lista de escolhas está vazia e encerra. Simples, e sem código extra.

## 6.6 A saída de segurança do capítulo 9

Vale contar isso porque foi um bug real, encontrado testando.

No capítulo 9 existem três formas de entrar no cemitério: com a chave, com a medalha, ou pulando o muro (que exige fôlego 65). Um jogador que chegasse ali sem nenhuma das três teria como **única** opção disponível a escolha que mata.

Ou seja: ele perderia o jogo sem ter feito nada de errado — só por não ter os itens. Isso não é dificuldade, é beco sem saída.

A correção foi acrescentar uma escolha sem nenhuma condição:

```
new Escolha("ESC0907", "Ficar no portão e esperar o sol nascer.", getCena("FIM_TARDE"))
    .comAtributo(Atributo.NERVO, -15)
```

### **A REGRA QUE VALE PARA O PROJETO INTEIRO**

**Toda cena precisa de pelo menos uma saída que não seja a morte.** Vale conferir isso em todas as cenas antes da entrega — e um teste automático que percorra as cenas verificando essa regra seria um excelente teste de unidade.

## 7. Requisito por requisito

O que o enunciado da Fase 1 pede e onde está no código.

| Requisito                                                          | Onde está                                                                                                                                    |
|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Menu inicial</b> com nova partida, instruções, créditos e saída | <code>MenuInicial.exibir()</code> e o <code>switch</code> do <code>JogoController.iniciarPartida()</code>                                    |
| <b>Configuração do protagonista</b>                                | <code>MenuInicial.pedirNome()</code> — o jogador informa o nome, que passa para o construtor da <code>Partida</code>                         |
| <b>Introdução narrativa</b>                                        | O texto do <code>CAP01</code> apresenta a cidade, o conflito e a situação                                                                    |
| <b>Ao menos 10 unidades identificáveis</b>                         | Dez capítulos, cada um com id próprio ( <code>CAP01</code> a <code>CAP10</code> ). <code>Historia.totalDeCenas()</code> devolve 28           |
| <b>Diálogos legíveis</b>                                           | Classe <code>Dialogo</code> + o <code>for-each</code> em <code>ExibirJogo</code> , no formato <code>Nome: texto</code>                       |
| <b>Escolhas em vários momentos</b>                                 | 55 objetos <code>Escolha</code> distribuídos pelas cenas                                                                                     |
| <b>Consequências imediatas</b>                                     | <code>Escolha.aplicar(partida)</code> — muda confiança, atributo, inventário, e a view mostra o que mudou                                    |
| <b>Consequências posteriores</b>                                   | As 13 flags: ligadas por <code>ligaFlag</code> e lidas por <code>exigeFlag</code> capítulos depois                                           |
| <b>Mínimo 5 personagens significativos</b>                         | Otávio, Davi, Seu Antônio, Dona Zulmira e Manuela — todos com fala e com confiança que varia                                                 |
| <b>Relacionamento com pelo menos 3</b>                             | <code>Npc.confianca</code> (0 a 100), alterada por escolhas e lida como condição em várias delas                                             |
| <b>3 atributos variáveis</b>                                       | <code>Protagonista</code> : <code>fôlego</code> , <code>nervo</code> , <code>lucidez</code> — alterados por <code>alterarAtributo</code>     |
| <b>Recurso gerenciável</b>                                         | <code>Inventario</code> com 9 itens                                                                                                          |
| <b>Condições de acesso</b>                                         | <code>Escolha.estaDisponivel(partida)</code> — cinco tipos: atributo, item, confiança, flag exigida e flag proibida                          |
| <b>Ao menos 3 finais + volta ao menu</b>                           | <code>FIM_A_TEMPO</code> , <code>FIM_TARDE</code> , <code>FIM_DESISTE</code> (mais 8 mortes). O laço do menu traz o jogador de volta sozinho |
| <b>Padrão MVC</b>                                                  | Pastas <code>model</code> , <code>view</code> e <code>controller</code> , com a direção das dependências respeitada                          |
| <b>Sem engine de jogo</b>                                          | Só bibliotecas padrão do Java ( <code>List</code> , <code>Map</code> , <code>Set</code> , <code>Scanner</code> )                             |

### O que ainda falta

Sendo honesto sobre o estado atual:

- **Testes de unidade.** O enunciado exige e ainda não existem. A boa notícia é que o desenho favorece: `estaDisponivel` e `aplicar` são as coisas mais fáceis de testar do

projeto.

- **Distribuir pontos na criação do personagem.** Hoje o jogador só digita o nome; os três atributos começam em 50. O enunciado fala em "informar ou selecionar características iniciais".

## Como seria o primeiro teste

```
// Verifica que a escolha 0505 só aparece se a flag PORTA_ABERTA estiver ligada

Partida sem = new Partida("Teste");
Historia h1 = new Historia();
h1.montarHistoria(sem);
// procura a ESC0505 no CAP05 -> estaDisponivel(sem) deve ser FALSE

Partida com = new Partida("Teste");
Historia h2 = new Historia();
h2.montarHistoria(com);
com.ligarFlag(Flag.PORTA_ABERTA);
// a mesma escolha -> estaDisponivel(com) deve ser TRUE
```

Esse teste foi executado durante o desenvolvimento e passou nos dois casos. Formalizá-lo com JUnit é o próximo passo.

## 8. Glossário

---

Todo termo técnico que aparece no projeto, em uma linha.

### | Java — a linguagem

| Termo                     | O que é                                                              |
|---------------------------|----------------------------------------------------------------------|
| <b>Classe</b>             | A descrição de um tipo de objeto. A planta da casa                   |
| <b>Objeto</b>             | Uma coisa concreta criada a partir de uma classe. A casa construída  |
| <b>Instanciar</b>         | Criar um objeto. É o que o <code>new</code> faz                      |
| <b>Atributo / campo</b>   | Uma variável que vive dentro de um objeto                            |
| <b>Método</b>             | Um bloco de código com nome, que o objeto sabe executar              |
| <b>Construtor</b>         | Método especial que roda quando o objeto nasce. Tem o nome da classe |
| <b>Parâmetro</b>          | O valor que um método recebe entre parênteses                        |
| <b>Retorno</b>            | O valor que um método devolve. <code>void</code> = não devolve nada  |
| <code>this</code>         | "Deste objeto aqui"                                                  |
| <code>super</code>        | "Da classe pai". <code>super(nome)</code> chama o construtor do pai  |
| <code>null</code>         | "Não aponta para nada". Um campo vazio                               |
| <code>static</code>       | Pertence à classe, não a cada objeto. Existe um só                   |
| <code>final</code>        | Não pode mudar depois de definido                                    |
| <b>Sobrecarga</b>         | Vários métodos com o mesmo nome e parâmetros diferentes              |
| <b>Pacote ( package )</b> | Uma pasta com classes relacionadas                                   |
| <code>import</code>       | Avisa que esta classe vai usar outra, de outro pacote                |

## Orientação a objetos

| Termo                         | O que é                                                             |
|-------------------------------|---------------------------------------------------------------------|
| <b>Encapsulamento</b>         | Esconder os dados (private) e expor só métodos. Protege as regras   |
| <b>Herança</b>                | Uma classe aproveita o que outra tem. <code>extends</code>          |
| <b>Classe abstrata</b>        | Uma classe que não pode ser instanciada. Só serve de base           |
| <b>Polimorfismo</b>           | Tratar objetos de tipos diferentes pelo mesmo tipo comum            |
| <b>Coesão</b>                 | Uma classe (ou método) faz uma coisa só, e bem                      |
| <b>Acoplamento</b>            | O quanto uma classe depende de outra. Quanto menos, melhor          |
| <b>Getter</b>                 | Método que só devolve o valor de um atributo                        |
| <b>Setter</b>                 | Método que só troca o valor de um atributo                          |
| <b>Injeção de dependência</b> | A classe recebe pronto o que precisa, em vez de criar               |
| <b>Interface fluente</b>      | Métodos que devolvem <code>this</code> para poder encadear chamadas |

## Estruturas de dados

| Termo                         | O que é                                                                              |
|-------------------------------|--------------------------------------------------------------------------------------|
| <b>List</b>                   | Fila ordenada. Aceita repetidos. Acesso por posição, a partir de zero                |
| <b>ArrayList</b>              | A implementação de <code>List</code> mais usada                                      |
| <b>Map</b>                    | Dicionário: guarda pares chave → valor                                               |
| <b>HashMap</b>                | A implementação de <code>Map</code> mais usada                                       |
| <b>Set</b>                    | Conjunto. Só responde "está dentro ou não". Sem repetidos                            |
| <b>HashSet</b>                | A implementação de <code>Set</code> mais usada                                       |
| <b>enum</b>                   | Lista fechada de valores possíveis, com nome                                         |
| <b>Genérico ( &lt;T&gt; )</b> | Diz o que cabe dentro da coleção. <code>List&lt;Escolha&gt;</code> só aceita Escolha |

## Do projeto

| <b>Termo</b>              | <b>O que é</b>                                            |
|---------------------------|-----------------------------------------------------------|
| <b>Cena</b>               | Um trecho da história: narração, falas e escolhas         |
| <b>Escolha</b>            | Uma alternativa, com condições e consequências            |
| <b>Flag</b>               | Uma decisão que o jogo memoriza para cobrar depois        |
| <b>Partida</b>            | O estado de um jogo em andamento                          |
| <b>Historia</b>           | A montadora do roteiro. Cria todas as cenas               |
| <b>Condição de acesso</b> | O que precisa ser verdade para uma escolha aparecer       |
| <b>Consequência</b>       | O que uma escolha muda no estado da partida               |
| <b>Duas passadas</b>      | Criar todas as cenas vazias e só depois ligar as escolhas |

## 9. Perguntas da banca

---

As perguntas prováveis na apresentação, com respostas curtas. Adapte para as suas palavras — respondida decorada aparece.

### "Por que `PersonagemBase` é abstrata?"

Porque "um personagem genérico" não existe no jogo: ou é o protagonista, ou é um NPC. Marcando abstrata, o compilador impede criar um objeto dela. É transformar um erro conceitual em erro de compilação.

### "Por que a confiança está no `Npc` e não no `PersonagemBase`?"

Porque o protagonista não tem confiança em relação a si mesmo. Um campo que não faz sentido para metade das subclasses subiu alto demais na hierarquia.

### "Por que a `Escolha` tem tantos campos?"

Porque são opcionais. Uma escolha usa dois ou três; o resto fica nulo e não é verificado. Foi a solução mais direta para atender aos requisitos de consequência e condição sem criar uma classe para cada tipo de efeito.

### "Onde fica a regra de que um atributo não passa de 100?"

Dentro do `alterarAtributo`, no método privado `limitar`. Como esse é o único caminho para alterar um atributo, não existe como um valor inválido entrar.

### "O que acontece se eu terminar o jogo e começar de novo?"

Uma `Partida` nova é criada, e o construtor dela cria NPCs novos, inventário novo e conjunto de flags vazio. Nada da partida anterior sobrevive.

### "Como vocês garantem o MVC?"

Pela direção das dependências: o controller conhece a view e o model; a view lê o model mas nunca o altera; o model não conhece ninguém. Não existe nenhum `System.out` dentro do model, nem `import controller` dentro da view.

### "Por que existe um Scanner só?"

Porque cada Scanner mantém um buffer interno. Com dois lendo o mesmo teclado, o primeiro pode consumir texto destinado ao segundo, e o segundo estoura com `NoSuchElementException`. E o bug depende de velocidade — digitando devagar funciona, alimentando rápido quebra, que é justamente como um teste automatizado alimenta.

### "Por que a história é montada em duas passadas?"

Porque o construtor de `Escolha` exige a cena de destino já existente. Numa passada só, seria preciso escrever os capítulos de trás para frente, e seria impossível uma cena apontar para trás.

## "Cadê a consequência posterior?"

Nas flags. A escolha `ESC0102`, no capítulo 1, liga `PORTA_ABERTA`. A escolha `ESC0505`, no capítulo 5, exige essa flag para aparecer. Quem não pediu a porta destrancada vê a opção bloqueada com o motivo "a porta está trancada".

## "Por que a `Historia` tem 900 linhas?"

Porque ela é o conteúdo: os textos dos dez capítulos estão nela. O enunciado cobra *métodos* coesos, e nenhum método dela passa de 45 linhas — a classe tem um método por capítulo.

## "O que é aquele `return this` nos métodos da `Escolha`?"

É o que permite encadear as chamadas. Cada método devolve o próprio objeto, então o próximo pode ser chamado em seguida. Sem isso, montar 55 escolhas exigiria uma variável e três linhas para cada uma.

## "Por que `enum` em vez de `String`?"

Porque um erro de digitação em `String` compila e falha silenciosamente em tempo de execução. Com `enum`, o erro aparece na hora de escrever. Além disso, o `enum` documenta quais são os valores possíveis.

## "O que acontece se o jogador digitar uma letra?"

O `nextInt()` lança `InputMismatchException`, que é capturada num `catch`. O programa limpa a entrada, avisa e pergunta de novo, num laço.

## "Como vocês testariam isso?"

Criando uma `Partida`, montando a história e verificando `escolha.estaDisponivel(partida)` antes e depois de ligar uma flag. Também dá para testar `aplicar` conferindo a confiança antes e depois. Nenhum dos dois precisa de console.

---

## Encerramento

Se tiver que guardar três frases deste documento inteiro, que sejam estas:

1. **Conteúdo e estado são coisas separadas.** A `Historia` é o livro; a `Partida` é o marcador de página e a mochila.
2. **A escolha carrega condições e consequências**, e sabe se aplicar sozinha. O controller só coordena.
3. **Uma flag ligada num capítulo e lida em outro** é o que transforma uma decisão em consequência posterior — o requisito mais difícil do enunciado.

---

Documento gerado a partir do código-fonte do projeto. Se o código mudar, vale regerar — diagrama e explicação valem pouco quando não batem com o que foi entregue.